ವಿಶ್ವೇಶ್ವರಯ್ಯ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ, ಬೆಳಗಾವಿ
VISVESVARAYA TECHNOLOGICAL UNIVERSITY - BELAGAVI

Department of Computer Science and Engineering

"Jnana Sangama", VTU-Campus, Belagavi-590018

B. TECH IN Artificial Intelligence & Data Science

Architectural Analysis of Spotify as a Distributed System

SUBJECT : DISTRIBUTED SYSTEM

SUBJECT CODE : BCS515D

GROUP NO. : 11

SUBMITTED BY

NAME	USN
1. Nageshwari G	2VX24AD058
2. Nafisa Asoti	2VX24AD057
3. Aishwarya G	2VX24AD008
4.Lovely kawad	2VX24AD047
5.Dhanshree	2VX24AD031

Architectural Analysis of Spotify as a Distributed System

Section 1: Introduction

Spotify is a cloud-hosted audio streaming service for music, podcasts and audiobooks. At the end of the second quarter of 2026 it reported **777 million monthly active users, 300 million of them paying Premium subscribers** [1], and by March 2026 it was present in 184 countries and territories [2]. It is also the backend for the desktop, mobile and web players, for smart speakers, cars and consoles, and for the many third-party apps that use its Web API. From a listener's point of view the requirements are simple: search a huge catalogue, press play and hear audio within a second, build and share playlists, download tracks for offline use, and carry the session from phone to laptop to car.

In distributed systems engineering we separate these **functional user features** (*what* Spotify does) from the **non-functional system goals** (*how* the infrastructure delivers them). The features are background context. The engineering interest lies in how Spotify achieves the following properties for hundreds of millions of concurrent listeners:

- **High availability and durability:** a track must start playing even when servers, zones or data centres fail, and a user's playlists and library must never be lost.
- **Horizontal scalability:** capacity and throughput grow by adding machines, not by buying bigger ones.
- **Consistency:** a playlist edited on one device, or by a collaborator, must look the same everywhere, and one account must not stream on two devices at once.
- **Low latency:** search results, track start and device switching must feel instantaneous worldwide.
- **Security, transparency and heterogeneity:** licensed audio protected on untrusted devices and networks, on thousands of device types, with all distribution hidden from the listener.

Scope note. *Spotify does not publish its complete internal design. This analysis is built from Spotify's own engineering blog and conference talks, Google Cloud's public case studies of the Spotify migration, the public Spotify Web API documentation, Spotify's quarterly filings, and standard distributed-systems literature. Where a mechanism is a documented Spotify practice, it is cited. Where it is an industry-standard technique that a system of this kind would use, the text says so and does not claim it as confirmed Spotify internals.*

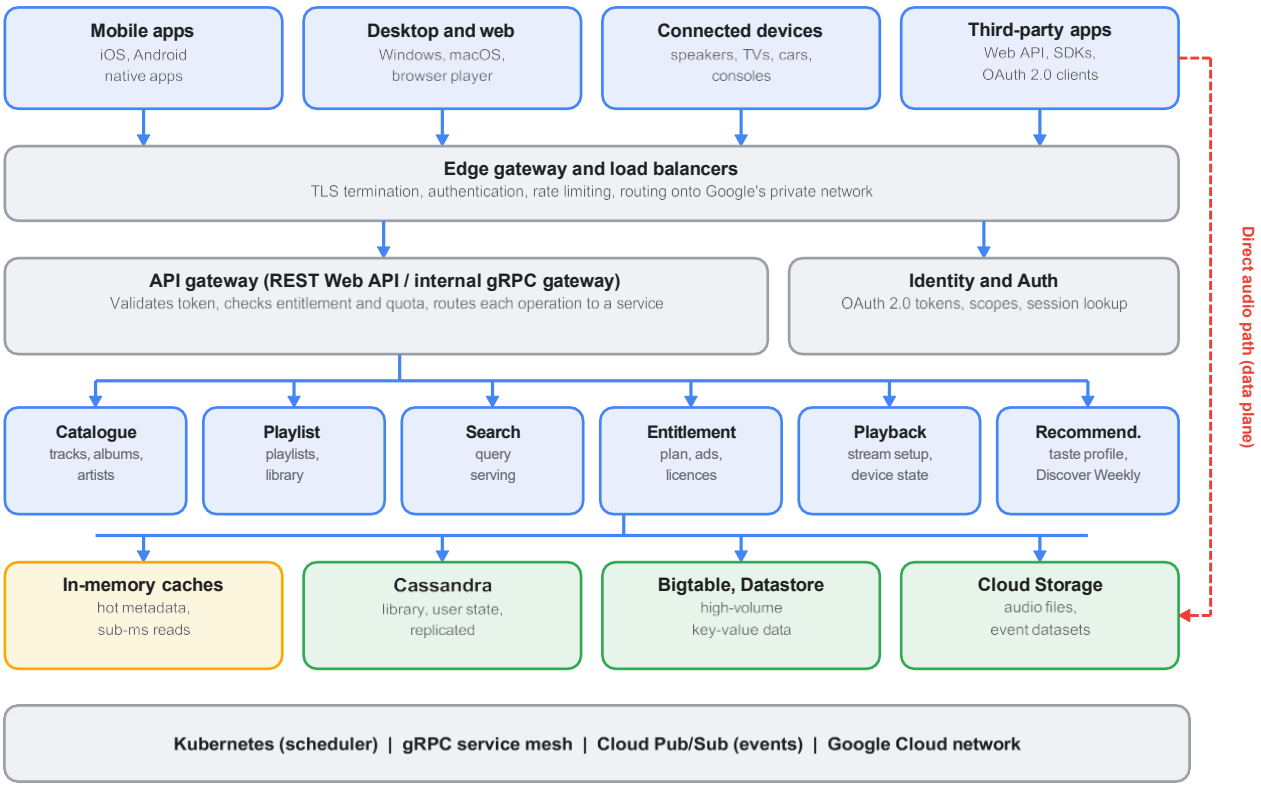
Section 2: System Architecture and Application Entities Mapping

Spotify is not one program on one server. It is a large collection of cooperating microservices running on Google Cloud. In 2016 Spotify announced its move from its own data centres to Google Cloud Platform [3], in two tracks, services and data. Nearly 1,200 microservices were moved, with traffic routed to the cloud by May 2017 [9]. The overall design separates two operating planes:

- **Control (metadata) plane:** catalogue metadata (artists, albums, tracks), user accounts, playlists, library, entitlements (free or Premium), search and recommendations. This data is small, structured and read constantly, so it lives in **replicated databases** and in-memory caches.
- **Data plane:** the audio bytes themselves. These are large and immutable once published, and identical for millions of listeners, so they are stored once as encoded files and delivered through **caches and a content-delivery network** close to the listener (an industry-standard design; the exact topology is not public).

  Delivery and caching layer: Frequently requested metadata and audio content can be served through caches content-delivery infrastructure. This helps reduce repeated requests to central services, lower network latency, and improve the listening experience.

FIGURE 1 : SPOTIFY OVERALL 5-TIER SYSTEM ARCHITECTURE



Detailed Breakdown of Application Entities:

- **Client tier:** the desktop app, iOS and Android apps, the web player, and connected devices (speakers, TVs, cars, consoles). Third-party apps use the public Spotify Web API. All clients are untrusted and heterogeneous.
- **Edge tier:** client traffic enters at an edge gateway that terminates TLS, authenticates the session, applies rate limits and forwards each request to the right backend service. The exact products are not public.
- **Identity and API gateway:** validates the OAuth 2.0 access token and its scopes, checks the account's entitlement and quota, and routes each operation to the right service.
- **Service tier (microservices):** many small, single-purpose services owned by autonomous squads: *Catalogue*, *Playlist* and library, *Search*, *Entitlement*, *Playback* and device state, and *Recommendation* services. Spotify's engineering talks describe more than 3,000 services calling each other over gRPC [8].
- **Storage tier (polyglot persistence):** in-memory caches, Apache Cassandra and Google Cloud Bigtable / Datastore for high-volume operational data, and Google Cloud Storage for large objects and datasets. Each engine is matched to its workload (Section 4).
- **Event and data tier:** every user action becomes an event published to Google Cloud Pub/Sub, processed with Dataflow and analysed in BigQuery [3][4]. This pipeline feeds recommendations such as Discover Weekly.
- **Cluster infrastructure:** Kubernetes restarts failed instances, and a service mesh built on Envoy's xDS APIs with proxyless gRPC handles service discovery and load balancing, giving dynamic traffic splitting and zone-aware routing [7].

Life of a track play (control plane and data plane working together):

1. The client opens an authenticated, encrypted connection to the edge, which checks the account's entitlement (Premium, free with ads, region).
2. The client asks the metadata and playback services to **resolve the chosen track** to the correct audio file for the device and quality setting.

3. The audio is fetched in **chunks** from nearby caches or the CDN; the first chunks start playback while later ones are still downloading.
4. The player decrypts and decodes the stream and prefetches the start of the next track so the gap is small (a common design for gapless playback).
5. When the track passes the play threshold, the client sends a **play event** that is published to Pub/Sub; it later feeds royalty reporting, charts and recommendations.
6. Other devices on the same account learn the new playback state through a persistent push connection, so the "now playing" bar stays in sync.

Section 3: Remote Invocation (RPC/RMI) and Concurrency Model

Remote communication between independent nodes requires explicit protocols, parameter marshaling, and server concurrency designs. Spotify uses different communication styles, one for the public boundary and one for the interior. Historically Spotify built services on its own proprietary messaging protocol and framework, then began a multi-year migration of its services to gRPC [8].

Dual Communication Protocol Strategy:

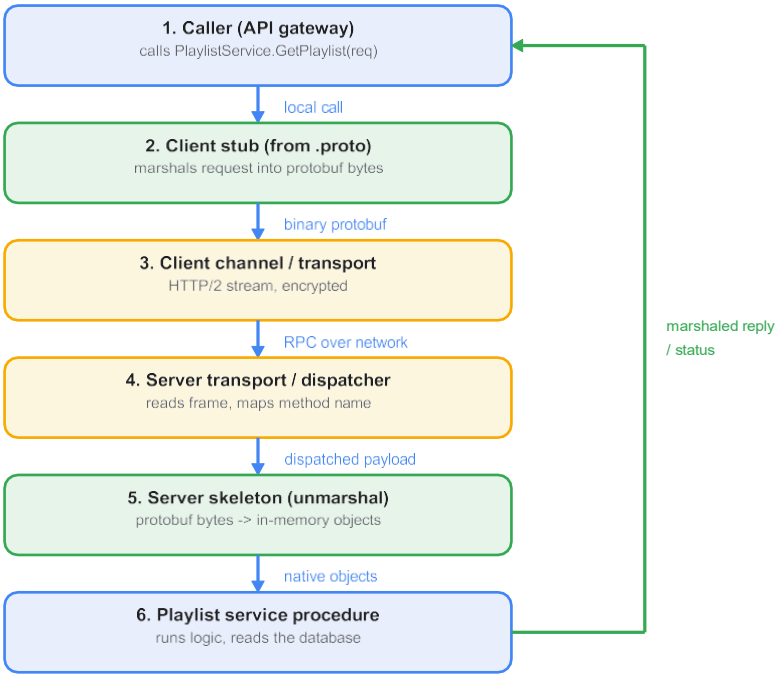
Boundary	Protocol and encoding	Reason
External (apps and third parties to the edge)	HTTPS with JSON bodies and OAuth 2.0 bearer tokens for the public Web API; persistent encrypted connections for the apps themselves	Universal support on every device and language; firewall-friendly; easy for third-party developers
Internal (service to service)	gRPC over HTTP/2 with **Protocol Buffers**	Compact binary payloads, typed interfaces, versionable schemas, multiplexed streams, deadlines and very low per-call overhead

RPC Parameter Marshaling Lifecycle:

When one Spotify service (say the API gateway) calls a method on another (the Playlist service), the following steps occur, shown in Figure 2:

- **Invocation:** the caller invokes an ordinary-looking method on a local **client stub** generated from the service's .proto definition. This is what gives *access transparency*.
- **Marshaling:** the stub serialises the request into compact Protocol Buffer bytes, independent of language or CPU byte order.
- **Network dispatch:** the bytes travel on a long-lived multiplexed HTTP/2 connection. Spotify's mesh resolves which instance to call using Envoy's xDS APIs with gRPC's native proxyless load balancing, so the call needs no extra proxy hop [7].
- **Unmarshaling and execution:** the server's dispatcher maps the method name to a **server skeleton**, which rebuilds native objects and runs the procedure (for example reading a playlist from the database).
- **Return path:** the result, or an error status, is marshaled and returned to the waiting stub, which hands it back to the caller.

FIGURE 2 : RPC / RMI MARSHALING EXECUTION LIFECYCLE



INVOCATION CALL SEMANTICS IN SPOTIFY

At-least-once + idempotent: catalogue reads, search, audio chunk fetches - safe to retry blindly.
Version-checked writes: a playlist change names the snapshot_id it was based on, so a stale write is rejected.
Deadlines: every gRPC call carries a deadline that propagates down the call chain; slow calls fail fast.

Invocation Call Semantics:

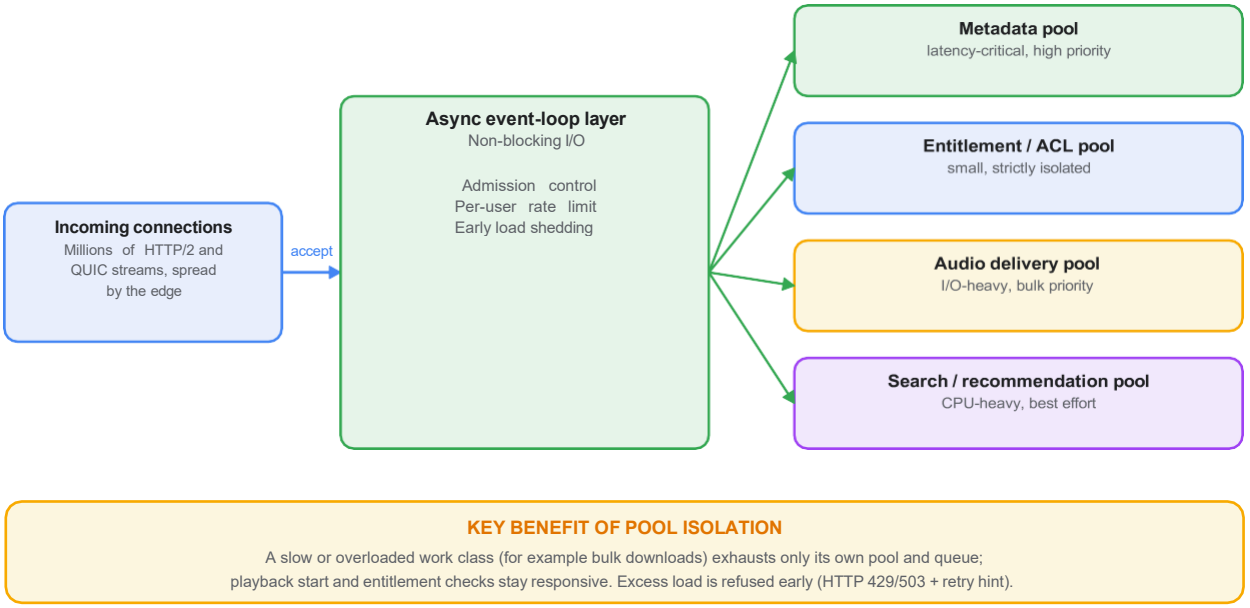
- **At-least-once with idempotency:** reads (catalogue lookups, search, fetching a playlist) and writes that set a value to a fixed state can be retried blindly, because repeating them leaves the same final state.
- **Optimistic preconditions for playlists:** the public Web API returns a *snapshot_id* (a version) for every playlist, and it can be sent back with a change so the server edits the version the client saw instead of silently overwriting a collaborator's work [11].
- **Deadlines:** gRPC lets each call carry a deadline that propagates down the call chain, so a slow dependency fails fast instead of piling up work. Spotify's gRPC rollout talks list deadlines and interceptors as key concepts [8].

Server Thread-Level Concurrency Model:

To serve very large numbers of simultaneous requests without starvation, large-scale servers combine several techniques. The details below describe widely used practice that a service like Spotify's follows; Spotify has not published its server internals.

- **Non-blocking, asynchronous I/O:** a few event-loop threads manage thousands of open connections and streams. A request waiting on a database does not hold a thread; its continuation runs when the reply arrives.
- **Work-class isolation:** different kinds of work are given separate pools and queues, so background work (analytics, bulk downloads) cannot crowd out latency-critical requests such as playback start and entitlement checks.
- **Admission control and rate limiting:** the public Web API rate-limits callers and answers excess calls with HTTP 429 plus a Retry-After header, so overload is refused early rather than queued until it times out [11].
- **Fine-grained parallelism:** a search fans out to index partitions and merges partial results, and a home screen is assembled from many services called in parallel, so one request uses many machines at once.

FIGURE 3 : ASYNCHRONOUS I/O AND ISOLATED WORKER-POOL CONCURRENCY



Section 4: Resource Sharing and Failure Handling Logic

Resource Allocation and Polyglot Persistence:

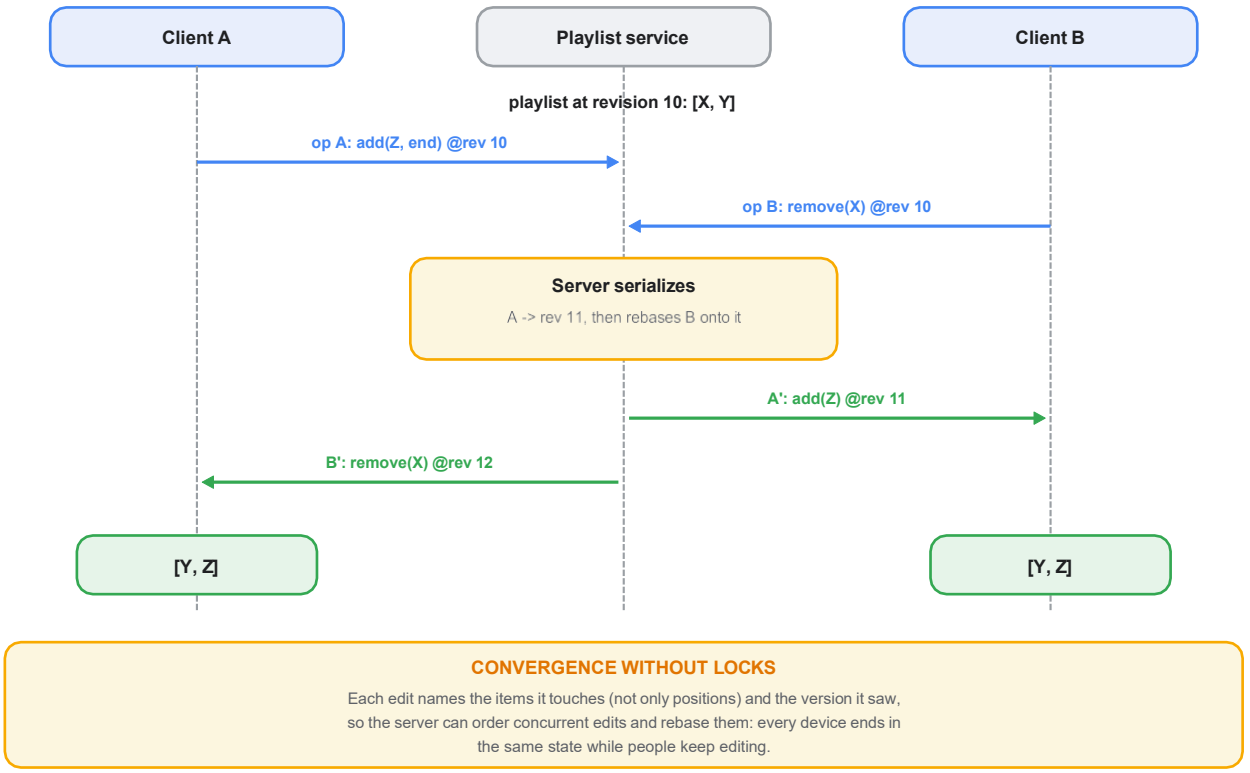
No single storage engine satisfies every Spotify requirement, so data is split across specialised resources. Spotify's 2016 cloud announcement named Cloud Datastore, Cloud Bigtable, Cloud Storage and BigQuery [3], and Spotify has also publicly described its use of Apache Cassandra. The exact mapping of each feature to each store is not public; the table lists the engines Spotify has named and the workloads they suit.

Store	What Spotify keeps there	Key property
In-memory caches	Hot catalogue metadata, recent search results, session and entitlement lookups	Sub-millisecond reads; loss is harmless (rebuilt from source)
Apache Cassandra	High-volume operational user data such as library and listening state	Replicated across nodes and data centres, tunable consistency, scales by adding nodes [15]
Bigtable / Datastore	Large key-value and entity data	Very high throughput, horizontal sharding by row key
Cloud Storage (GCS)	Large objects and datasets, including the event delivery system's final datasets [4]	Durable object storage, redundant across failure domains
BigQuery	Analytics over listening events	Columnar warehouse for large analytical queries

Sharing a Resource among Many Users:

- **One stream per account:** a Premium account plays on one device at a time. Starting playback on a second device pauses the first, so the account's "active device" is a small piece of shared state that must be updated consistently.
- **Playlists and collaboration:** a playlist can be edited by several people. Each playlist carries a version (the snapshot_id in the Web API), and edits are applied against a specific version. This is **optimistic concurrency control**: no locks, and a stale write is detected rather than silently lost (Figure 4).
- **Entitlements:** whether a user hears ads, can download or can pick any track depends on the plan, checked on every playback request so that a lapsed subscription takes effect promptly.
- **Offline downloads:** Premium users can download encrypted copies to a device. Playback from these copies needs no network, but the app must reconnect periodically so the licence can be revalidated.

FIGURE 4 : REVISION-BASED MERGING OF CONCURRENT PLAYLIST EDITS



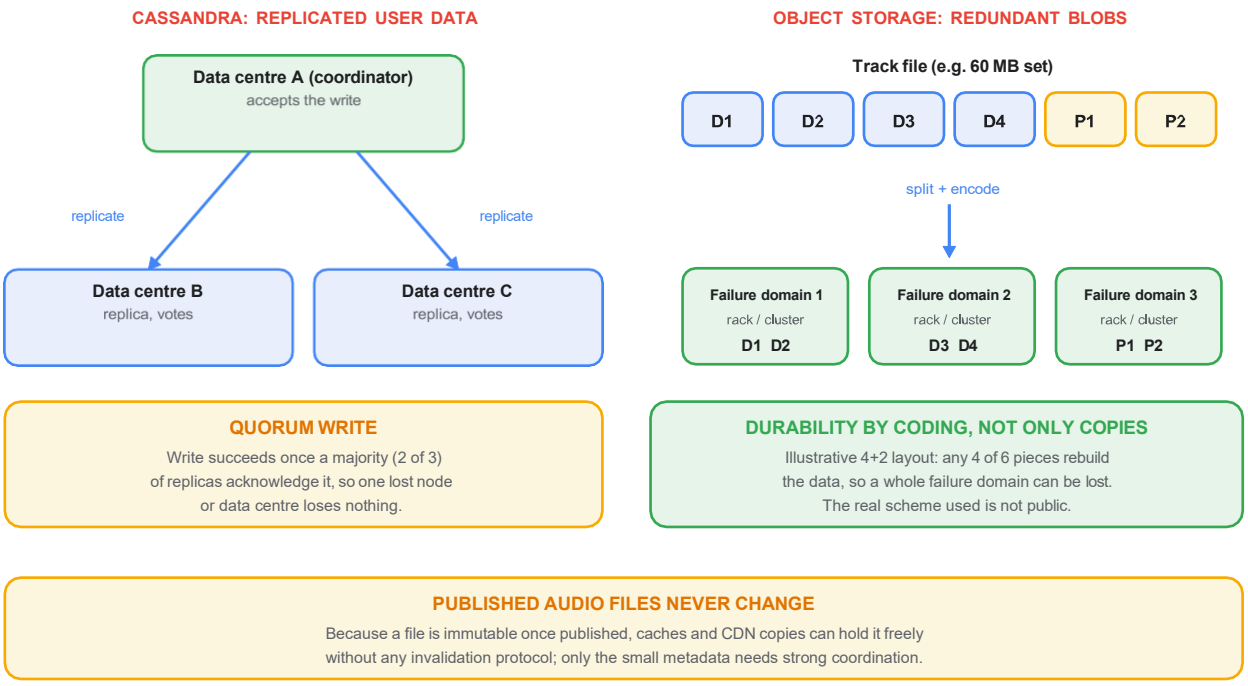
Illustrative model; Spotify's exact merge algorithm is not public.

Replication and Durability:

Two complementary techniques protect Spotify data. For metadata and user data, each piece is kept on several replicas in different locations. In a quorum-replicated store such as Cassandra, a write succeeds once enough replicas acknowledge it, so the loss of one node or one data centre neither stops writes nor loses data.

For audio files and datasets, storing many full copies would be costly at Spotify's scale. Cloud object stores instead keep data redundantly across failure domains, commonly by erasure coding: the data is split into chunks plus parity pieces, so missing pieces can be rebuilt from survivors. The coding scheme actually used is not public; Figure 5 shows an illustrative 4+2 layout. Because a published audio file never changes, it can be cached freely at many layers without invalidation problems.

FIGURE 5 : QUORUM REPLICATION AND ERASURE-CODED OBJECT STORAGE

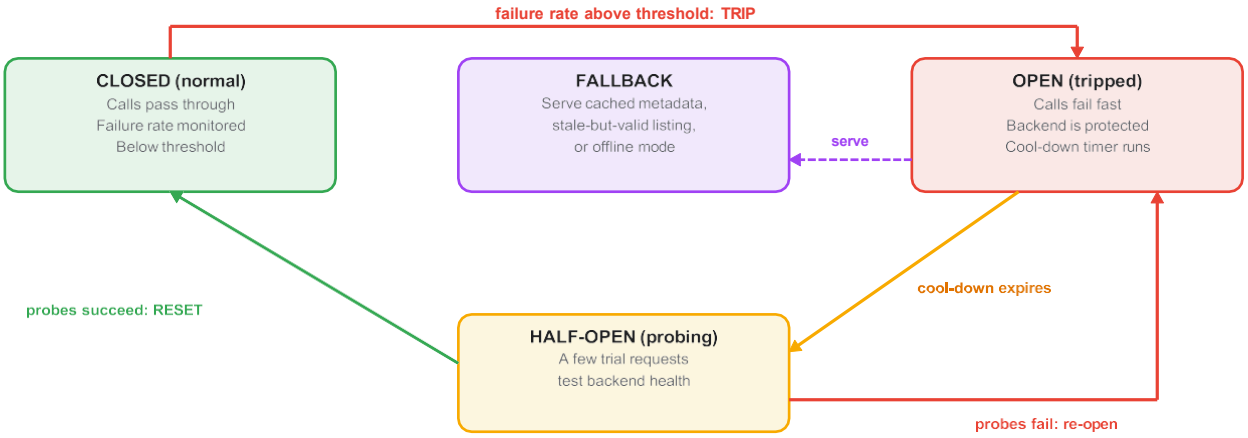


Fault Tolerance and Recovery Mechanisms:

In a system with thousands of services on a very large fleet, component failure is a constant, not an exception. Stability is maintained through several patterns:

- **Automatic rescheduling:** Kubernetes detects failed pods and nodes and restarts the work elsewhere; because services keep durable state in the databases, a restart loses nothing.
- **Retries, deadlines and failover:** calls carry deadlines, and a failed call can be retried on another replica or instance. Spotify's mesh adds zone-aware routing and fast fallback mechanisms so a bad rollout can be reverted quickly [7].
- **Circuit breaking and load shedding:** when a downstream service fails repeatedly, callers stop sending it traffic (the breaker opens) so it can recover, then probe it gently before resuming (Figure 6). The Google SRE literature describes these as core defences against cascading failure [14].
- **Graceful degradation:** a non-essential failure should reduce quality, not availability. If recommendations are unavailable, search and playback still work; if the network drops, the client keeps playing from its buffer and from downloaded tracks.
- **Isolation of event processing:** in the event pipeline each event type has its own Pub/Sub topic, ETL process and storage location, so a failure in one event type does not block the others [4].

FIGURE 6 : CIRCUIT-BREAKER STATE MACHINE WITH FALLBACK



Thresholds and timers are illustrative; real values are tuned per service.

Section 5: Message Passing and Reliability Mechanisms

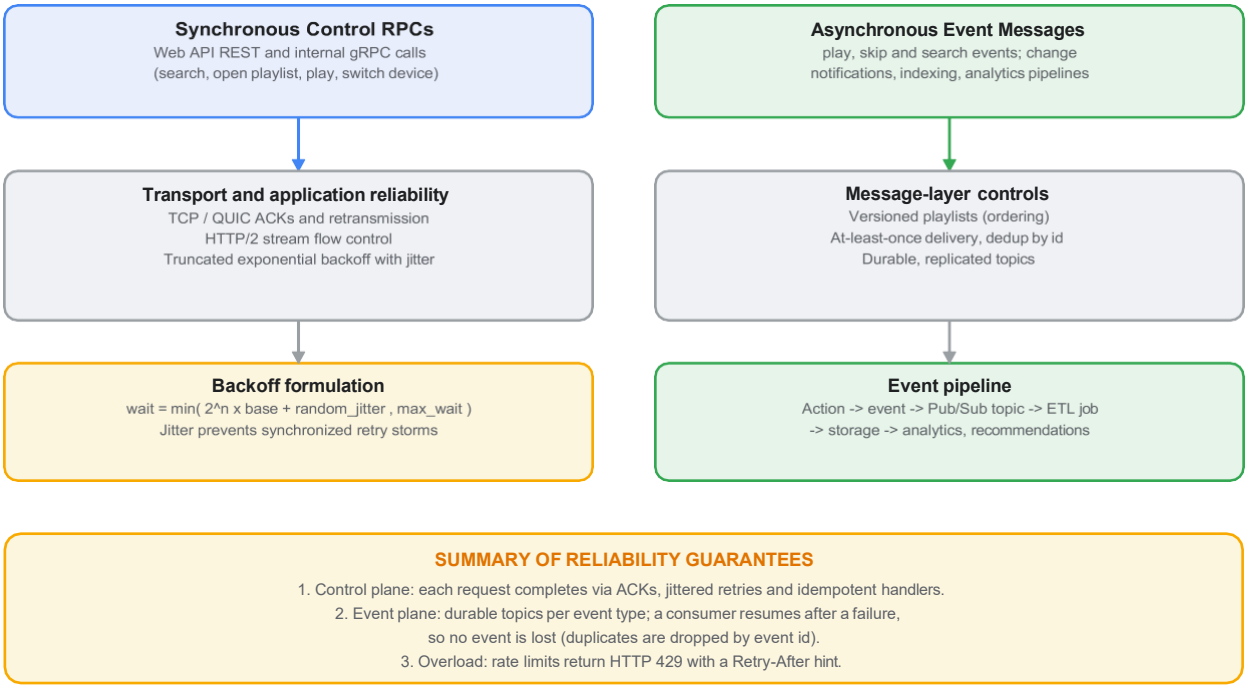
Explicit Message Classifications:

Communication across Spotify falls into two message types:

- **Synchronous control messages:** request-reply exchanges (REST or internal gRPC) for immediate user actions such as searching, opening a playlist, starting playback or switching device.
- **Asynchronous event messages:** a durable stream of events. Whenever a user plays, skips, searches or shares, an event is published and processed later without slowing the user's request. Spotify's event delivery system handled more than 8 million events per second at peak and over 350 TB of raw event data per day at the end of Q1 2019 [6].

The event pipeline was redesigned for the cloud. The earlier on-premises system was built on Kafka 0.7 and hourly files; Spotify chose Cloud Pub/Sub as the transport after a load test at Spotify scale [5], and the old Kafka-based system was turned off in February 2017 [6]. Each event type has its own topic, ETL process and final storage, so one noisy or broken event type cannot delay the rest [4].

FIGURE 7 : MESSAGE-PASSING PIPELINE AND RELIABILITY CONTROLS



Message-Layer Reliability Mechanisms:

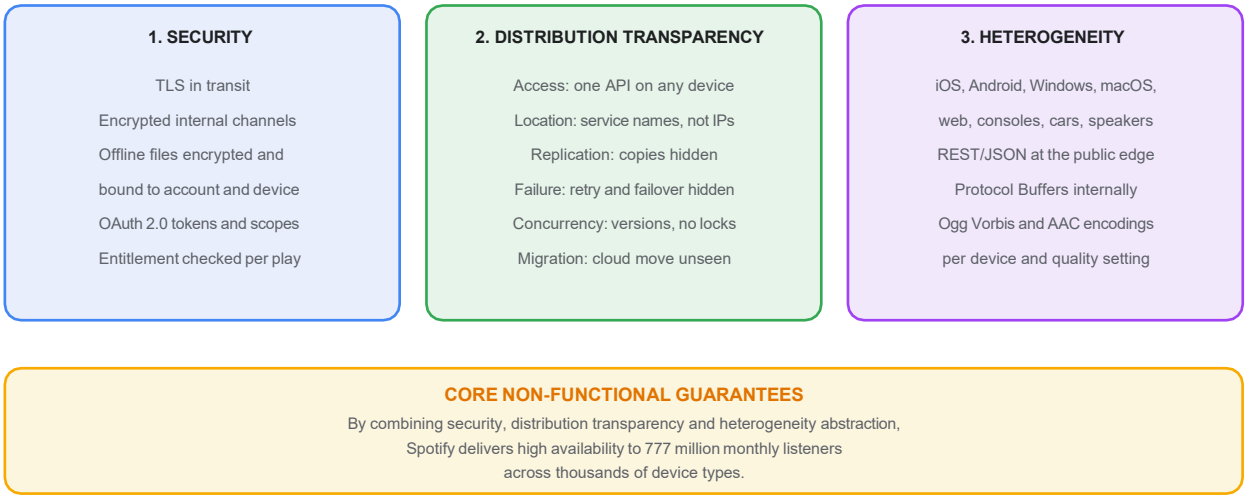
To guarantee delivery and prevent data loss over unreliable networks (transport-level items are standard internet protocols; where a Spotify-specific source exists it is cited):

- **Positive acknowledgements and retransmission:** TCP and QUIC require the receiver to acknowledge data and resend what is not acknowledged. In Pub/Sub a message stays queued until the consumer acknowledges it, and is redelivered otherwise.
- **At-least-once delivery with deduplication:** because a message can be redelivered, consumers must tolerate duplicates, usually by giving each event a unique id and discarding repeats.
- **Exponential backoff with jitter:** after the n-th failure a client waits about 2^n × base plus random jitter, up to a maximum. Jitter stops thousands of clients retrying in lockstep and overwhelming a recovering server (the thundering-herd problem). The Web API's 429 response with a Retry-After header tells callers how long to wait [11].
- **Ordering:** TCP sequence numbers restore packet order, and audio chunk byte ranges restore file order. For playlists, a version number gives one order to changes, so a client applying edits in sequence reaches the same state as the server.
- **Flow control:** HTTP/2 and TCP window mechanisms stop a fast sender from overwhelming a slow receiver, and server-side rate limiting provides the same protection at the application layer.
- **Integrity:** encoded audio is verified by the player and the transport, so clients can detect corrupted chunks and fetch them again. This is a standard technique and not a confirmed Spotify internal.

Section 6: Core Distributed Systems Characteristics

This section evaluates how the main theoretical characteristics of distributed systems are realised in Spotify [13].

FIGURE 8 : SECURITY, TRANSPARENCY AND HETEROGENEITY IN SPOTIFY



1. Security Architecture:

- **Transport encryption:** all traffic between user devices and Spotify's edge uses TLS, and internal service calls run on encrypted HTTP/2 channels.
- **Content protection:** licensed audio is delivered in protected form. Offline copies are encrypted and bound to the user's account and device, so a file copied elsewhere cannot be played.
- **Authentication and authorisation:** the Web API uses OAuth 2.0 access tokens with narrowly defined scopes, so a third-party app sees only what the user approved [11]. Entitlement checks run on playback requests.

2. Distribution Transparency:

- **Access transparency:** the same operations work from a phone, laptop, car or speaker, and stubs hide all remote-call detail.
- **Location transparency:** services call each other by logical names resolved by the service mesh, and listeners never see which data centre serves them.
- **Replication and failure transparency:** database replicas, redundant object storage and cached audio are invisible to callers, and instance or zone failures are absorbed by retries and failover.
- **Concurrency transparency:** versioned playlists and one-active-device rules let many people and devices share an account without ever seeing locks.
- **Migration transparency:** Spotify moved about 1,200 services to the cloud while keeping the product running [12], so users did not notice.

3. Support for Heterogeneity:

Spotify runs on iOS, Android, Windows, macOS, Linux, web browsers, game consoles, cars and speakers, with very different CPUs and codecs. It hides these differences behind platform-neutral REST/JSON at the public edge and language-neutral Protocol Buffers internally. Audio is encoded in several formats so each device gets a suitable one: according to Spotify support, uploaded FLAC or WAV masters are transcoded to Ogg Vorbis at 96, 160 and 320 kbps and to AAC at 128 and 256 kbps [10].

4. Scalability and Openness:

Scalability comes from small, independently scaled services and sharded storage: Cassandra and Bigtable split data by key, and Pub/Sub absorbs millions of events per second. Spotify's service-mesh talk describes a scale of about 2 million Kubernetes pods [7]. Openness comes from the published, versioned Web API and OAuth scopes, which let third-party apps and device makers extend Spotify without access to its internals.

Summary: Characteristic-to-Mechanism Mapping

Characteristic	Mechanism in Spotify
Availability	Replicated databases, Kubernetes rescheduling, zone-aware routing, graceful degradation, buffered and offline playback
Durability	Redundant object storage, replicated user data, immutable audio files
Scalability	Thousands of independent microservices, sharded Cassandra and Bigtable, Pub/Sub for events, CDN and caches
Consistency	Versioned playlists (optimistic concurrency), one active device per account, replicated stores with tunable consistency
Low latency	Edge gateways, in-memory caches, CDN delivery, chunked streaming, prefetching, proxyless gRPC
Fault tolerance	Retries with backoff and jitter, deadlines, circuit breakers, per-event-type isolation, fast rollout fallback
Security	TLS, encrypted offline files, OAuth 2.0 scopes, entitlement checks
Transparency and heterogeneity	Stubs and logical names; REST/JSON at the edge; Protocol Buffers internally; multiple audio codecs

Section 7: Conclusion

Spotify shows how a familiar consumer product depends on distributed-systems principles at every layer. A clean separation of metadata and audio data lets each be served by the system best suited to it; typed RPC and stubs hide the network; deadlines, isolation and circuit breakers keep local failures local; replication and redundant storage protect data from hardware loss; and versioned playlists let people share and edit without locks. The result is a service that stays available and responsive for 777 million monthly users while presenting one simple player on thousands of device types.

References

[1] Spotify Technology S.A. Q2 2026 Update, Form 6-K Exhibit 99.1, 4 August 2026 (sec.gov).

[2] Spotify Technology S.A. Form 6-K for the quarter ended 31 March 2026 (sec.gov).

[3] Google Cloud blog. Spotify chooses Google Cloud Platform to power data infrastructure, 2016 (cloud.google.com).

[4] Spotify Engineering. Spotify's Event Delivery: Life in the Cloud, 2019 (engineering.atspotify.com).

[5] Spotify Engineering. Spotify's Event Delivery: The Road to the Cloud, Part II, 2016 (engineering.atspotify.com).

[6] InfoQ. Experience Running Spotify's Event Delivery System in the Cloud, 2019 (infoq.com).

[7] Lindblad, E. and Manno, E. How We Moved Spotify to a Proxyless gRPC Service Mesh. KubeCon + CloudNativeCon Europe, 2025 (kccnceu2025.sched.com).

[8] Grüter, M. The (updated) story of why we migrate to gRPC and how we go about it. LeadDev, 2023 (leaddev.com).

[9] The Epic Migration of Spotify to Google Cloud Platform: 100+ teams, 1200 services (conference report, startupgames.substack.com).

[10] Spotify Community. Audio formats and bitrates for streaming, support reply (community.spotify.com).

[11] Spotify for Developers. Spotify Web API documentation: authorization, rate limits, playlists and snapshot_id (developer.spotify.com/documentation/web-api).

[12] Data Center Dynamics. Spotify tunes in to Google's cloud platform, 2016 (datacenterdynamics.com).

[13] Coulouris, G. et al. Distributed Systems: Concepts and Design. Pearson.

[14] Beyer, B. et al. Site Reliability Engineering: How Google Runs Production Systems. O'Reilly, 2016.

[15] Lakshman, A. and Malik, P. Cassandra: A Decentralized Structured Storage System. ACM SIGOPS Operating Systems Review, 2010.